AKADEMIA HANDLOWA NAUK STOSOWANYCH W RADOMIU



RADOM ACADEMY OF ECONOMICS

INSTRUKCJA LABORATORYJNA NUMER 1
Przedmiot: Zaawansowane programowanie internetowe
Cel praktyczny zajęć: Testowanie mechanizmów HTTP, OData i WebSocket w projektach z Demo.rar oraz przygotowanie krótkiej aplikacji Angular + ASP.NET Core do obsługi CRUD studentów.

Pracuj na projektach dostępnych w repozytorium: https://github.com/kicbar/ahns-zpi/tree/main/projects/Demo. Uruchom demonstracje, zanotuj obserwacje w DevTools i debuggerze, a następnie wykonaj krótki samodzielny projekt CRUD studentów. Do każdego testu sformułuj wnioski własnymi słowami (2–3 zdania).

1. Przygotowanie stanowiska i dokumentowanie

- Rozpakuj Demo.rar i otwórz Demo/Demo.slnx w Visual Studio. Projekt celuje w .NET 10 (wymagany odpowiedni SDK).
- W terminalu przejdź do katalogu Demo; uruchamiaj jeden projekt demonstracyjny na raz. Porty wynikają z profili http w plikach launchSettings.json.
- Otwórz F12 → Network. Rejestruj URL, metodę, status, nagłówki i treść odpowiedzi; dla WebSocket skorzystaj z zakładki Messages/Frames.
- Na potrzeby analizy kodu użyj breakpointów w Program.cs. Odróżnij rejestrację trasy/obsługi zdarzenia od wykonania handlera.
- Do sprawozdania dodaj krótki opis kroków, obserwacje, odpowiedzi i czytelne zrzuty. Nie trzeba przepisywać kodu demonstracyjnego.

Uwaga o zawartości archiwum: Demo.rar zawiera RestApi.Demo, OData.Demo i WebSocket.Demo. Nie ma w nim projektów Polling.Demo, Sse.Demo ani Webhook. wymienionych w starszej instrukcji. Demonstracje korzystają z danych w pamięci; nie udostępniają pełnego CRUD ani gotowego panelu użytkownika.*

2. Zadanie 1 — HTTP Request–Response (RestApi.Demo)

```
dotnet run --project RestApi.Demo --launch-profile http
```

Adres: http://localhost:5047/ | Endpoint: GET /

- Otwórz adres w przeglądarce. Odczytaj rekord JSON: Id, FirstName, LastName i Age (przykładowy student John Smith, 22 lata).
- W DevTools → Network znajdź GET /. Zapisz kod statusu, Content-Type i odpowiedź JSON.
- Odśwież widok trzy razy. Sprawdź, ile osobnych żądań GET powstało i czy każde ma własną odpowiedź.
- Ustaw breakpoint w handlerze MapGet w Program.cs. Sprawdź, kiedy uruchamia się handler; nie myl jego wykonania z rejestracją trasy podczas startu.

Dowód wykonania: jeden zrzut GET / z Network i informacja o statusie oraz formacie odpowiedzi.

Pytania do krótkich wniosków:

- 1. Dlaczego samo odświeżenie strony generuje nowe żądanie?
- 2. Czy pokazany projekt realizuje pełny CRUD? Jakich metod mu brakuje?
- 3. Które elementy należą do request, a które do response?

3. Zadanie 2 — OData: filtrowanie danych (OData.Demo)

```
dotnet run --project OData.Demo --launch-profile http
```

Adres: http://localhost:5089/odata/students | Gotowe przykłady: OData.Demo/odata-students.http

- Wywołaj GET /odata/students i zapisz liczbę studentów oraz dostępne pola modelu: Id, FirstName, LastName, Age.
- Uruchom kolejno poniższe zapytania (przeglądarka, plik .http lub klient HTTP):

```
GET /odata/students?$filter=Age gt 21
GET /odata/students?$orderby=Age desc
GET /odata/students?$select=FirstName,LastName
```

- Dla każdego zapytania określ, które rekordy lub pola zostały zwrócone. Zwróć uwagę na pisownię właściwości.
- W Program.cs odszukaj AddOData(...), AddODataQueryEndpointFilter() oraz WithODataResult(). Wskaż, który fragment odpowiada za obsługę składni OData.
- Sprawdź, czy projekt korzysta z SQL Server/EF Core czy z listy w pamięci. Nie zakładaj, że OData automatycznie wymaga bazy danych.

Dowód wykonania: odpowiedzi dla zapytania z $filter i $select oraz krótki opis działania $orderby.

Pytania do krótkich wniosków:

- 1. Czym różni się parametryzacja OData od ręcznie napisanego filtra w zwykłym API?
- 2. Czy OData.Demo wykonuje zapytania SQL?
- 3. Jakie zagrożenia mogłoby mieć udostępnienie nieograniczonego filtrowania klientowi?

4. Zadanie 3 — WebSocket: komunikaty dwukierunkowe (WebSocket.Demo)

```
dotnet run --project WebSocket.Demo --launch-profile http
```

HTTP host: http://localhost:5150 | Serwer WebSocket: ws://localhost:8181

W aktualnym Demo.rar nie ma gotowego panelu Connect/Send. Strona HTTP na porcie 5150 nie musi prezentować interfejsu. Do testu użyj konsoli JavaScript przeglądarki otwartej na dowolnej stronie lokalnej albo klienta WebSocket.

- W DevTools → Console wykonaj poniższy kod (ewentualnie użyj testowego klienta WebSocket):

```
const ws = new WebSocket("ws://localhost:8181");
ws.onmessage = e => console.log("SERWER:", e.data);
ws.onopen = () => { ws.send("Test 1"); ws.send("Test 2"); };
```

- W Network wybierz WS. Sprawdź, czy połączenie zostało zestawione i czy w Messages/Frames widać wiadomości wysłane oraz echo serwera.
- Wyślij trzeci komunikat komendą ws.send("Test 3"). Odczytaj odpowiedź rozpoczynającą się od „Received echo message:”.
- W Program.cs ustaw breakpoint wewnątrz socket.OnMessage i sprawdź, jaką wartość ma message przed socket.Send(...).
- Wykonaj ws.close(), a następnie spróbuj ponownie użyć ws.send(...). Zapisz rezultat i wyjaśnij rolę stanu połączenia.

Dowód wykonania: zrzut Messages/Frames zawierający komunikat klienta i odpowiedź serwera.

Pytania do krótkich wniosków:

- 1. Dlaczego kolejne send() nie tworzą osobnych żądań HTTP GET/POST?
- 2. Dlaczego projekt wykorzystuje porty 5150 i 8181?
- 3. Czym różni się WebSocket od jednorazowego wywołania HTTP?

5. Zestawienie mechanizmów i wnioski

Uzupełnij tabelę na podstawie wykonanych doświadczeń.

Mechanizm	Jak inicjujesz?	Co widzisz w Network?	Typowe zastosowanie
HTTP GET			
OData przez HTTP			
WebSocket			

Napisz 5–8 zdań porównania. Uwzględnij, czy komunikacja wymaga nowego requestu, czy utrzymuje połączenie, skąd pochodzą dane i czy klient może określać sposób ich filtrowania.

6. Projekt praktyczny — CRUD studentów (Angular + ASP.NET Core)

Zaprojektuj i zbuduj prostą aplikację do zarządzania listą studentów. Projekt jest oddzielny od demonstracji: nie przerabiaj trzech pokazowych projektów na gotowy system. Możesz wykorzystać ich mechanizmy jako inspirację. Baza SQL nie jest częścią wymagań tego laboratorium.

6.1. Wymagane technologie i model danych

- Frontend: Angular z TypeScript i HttpClient. Backend: ASP.NET Core Web API (C#). Dane studentów przechowuj w statycznej liście (List<Student>) w pamięci serwera LUB w lokalnym pliku JSON. SQL Server, EF Core i migracje nie są wymagane.
- Encja Student musi mieć co najmniej: Id (int, klucz główny), FirstName (string), LastName (string), Age (int). Można dodać FieldOfStudy lub Email.
- Dodaj co najmniej 3 przykładowych studentów. Każda operacja CRUD musi odczytywać i aktualizować wybrane źródło danych. Wariant z listą: dane mogą zniknąć po restarcie backendu. Wariant z plikiem JSON: zmiany powinny przetrwać restart.

6.2. Wymagane endpointy REST

Metoda i adres	Działanie	Oczekiwany rezultat
GET /api/students	Lista studentów	200 OK + tablica JSON
GET /api/students/{id}	Szczegóły studenta	200 OK albo 404
POST /api/students	Dodanie studenta	201 Created + identyfikator
PUT /api/students/{id}	Edycja całego studenta	200/204 albo 404
DELETE /api/students/{id}	Usunięcie studenta	204 albo 404

Opcjonalnie: PATCH /api/students/{id} do częściowej aktualizacji danych. Nie zastępuje obowiązkowego PUT.

6.3. Wymagane widoki Angular

- Widok listy studentów: tabela z Id, imieniem, nazwiskiem, wiekiem oraz przyciskami „Szczegóły”, „Edytuj”, „Usuń”.
- Widok dodawania: formularz FirstName, LastName i Age; zapis przez POST oraz powrót do listy po sukcesie.
- Widok edycji: pobranie studenta po Id, wypełnienie formularza i zapis zmian przez PUT.
- Widok szczegółów: prezentacja danych wskazanego studenta pobranych przez GET /api/students/{id}.
- Usuwanie: przycisk z potwierdzeniem, żądanie DELETE i odświeżenie listy po sukcesie.
- Obsłuż stan ładowania, pustą listę, błąd komunikacji z API i komunikat po udanym zapisie/usunięciu.

6.4. Walidacja i podstawowe wymagania jakościowe

- FirstName i LastName są wymagane; Age powinno należeć do sensownego zakresu (np. 16–100). Walidację wykonaj na froncie i w backendzie.
- Zastosuj poprawne statusy HTTP i zwracaj czytelne błędy. Nie dopuszczaj do utworzenia rekordu z niepoprawnymi wartościami.
- Oddziel komponenty widoków od serwisu komunikacji HTTP; po stronie .NET oddziel model danych od DTO żądań, jeśli to potrzebne.

– W README opisz wymagane narzędzia, komendy uruchomienia frontend/backend oraz wybrany sposób przechowywania danych (lista lub plik JSON). Jeśli używasz pliku, podaj jego lokalizację i sposób inicjalizacji.

6.5. Kryteria odbioru — sprawdź przed oddaniem

– □ Dodanie studenta powoduje pojawienie się nowego rekordu na liście bez ręcznej modyfikacji źródła danych.

– □ GET po Id pokazuje poprawne szczegóły, a nieistniejący Id daje 404.

– □ Edycja aktualizuje dane widoczne na liście i w szczegółach.

– □ Usunięcie usuwa rekord i aktualizuje widok listy.

– □ Błędny formularz nie prowadzi do zapisu; użytkownik widzi komunikat.

– □ Backend realizuje CRUD na liście w pamięci lub w pliku JSON. Przy liście opisano utratę danych po restarcie; przy pliku sprawdzono ich zachowanie po restarcie.

6.6. Rozszerzenia dla chętnych

– Filtrowanie i sortowanie listy; paginacja; PATCH; wykorzystanie OData na osobnym endpointzie; testy integracyjne API; dodatkowe pola studenta.

7. Sposób oddania

– Sprawozdanie: ZPI_LAB_01_imie_nazwisko.docx lub PDF — odpowiedzi i wnioski z zadań 1–3, tabela porównawcza, 3–5 opisanych zrzutów Network/Console.

– Projekt CRUD: repozytorium GitHub lub archiwum ZIP z kodem Angular + .NET i README; jeśli wybrano zapis w pliku, dołącz przykładowy plik JSON. Nie dołączaj katalogów bin/obj/node_modules ani danych poufnych.

– Do projektu dodaj 4–6 zrzutów: lista studentów, dodawanie, edycja, szczegóły, wynik usuwania i przykład walidacji; dołącz krótki opis testów CRUD.

– Nazwy plików: ZPI_LAB_01_imie_nazwisko oraz ZPI_CRUD_Studenci_imie_nazwisko. Sposób i termin przekazania zgodnie z ustaleniami prowadzącego.

Zakres Demo.rar zweryfikowany z plikami Program.cs i launchSettings.json: RestApi.Demo ma wyłącznie GET /; OData.Demo pracuje na List<Student>; WebSocket.Demo udostępnia serwer Fleck na ws://localhost:8181 bez dostarczonego panelu frontend.